# ECE391 Review

Author: Members of HKN

Questions and solutions created for the file system (NGFS) on October 25, 2026. This file is to just track progress on creating material for HKN's ECE391 worksheets. Since this is the first ever iteration for this class, I'm not going to adhere to a general structure for midterms and etc. That changes per semester anyways, so I'll just have the general topics covered starting in MP3 CP1 and onwards. These questions will need review!

## 1 File System: NGFS

(a) **Building the NGFS picture**: An NGFS device reserves its first $F = 3$ blocks for the FAT.

(i) Sketch the FAT and data regions. Where does the root directory begin? Give both its device block address and its data block address. Does NGFS support subdirectories?

(ii) What do `name`, `size`, and `start_blk` describe in a directory entry? What is special about the first entry, named `.`?

(iii) Explain the meanings of `FAT[i] = NGFS_BLOCK_FREE`, `FAT[i] = NGFS_BLOCK_END`, and `FAT[i] = 7`.

**Solution**

(i) Device blocks 0–2 contain the FAT. The data region begins at device block 3, which is data block 0 and the start of the root directory. In general, device block address = $F+$ data block address. NGFS is flat: the root is its only directory.

(ii) The fields give a file's name, exact byte length, and first data block address. The root directory is itself a file containing an array of entries. Its first entry describes itself: name `.`, starting data block 0, and size equal to the byte length of its entries, including `..` The directory can grow into a multi-block FAT chain.

(iii) Entry $i$ describes data block $i$. FREE (0) means available; END ($2^{32} - 1$) means the last block in a file; 7 means the next file block is data block 7. The links are in the FAT, not embedded in file contents.

(b) **Following a file**: A directory entry for `notes.txt` has `size = 1300` and `start_blk = 2`. The relevant FAT entries are:

```
FAT[2] = 7;
FAT[7] = 4;
FAT[4] = NGFS_BLOCK_END;
```

The FAT occupies three device blocks. The file position is initially 500.

(i) A caller requests 600 bytes. Identify each data block used, its device block address, and the number of bytes copied from it. What is the new position?

(ii) Instead, the position is 1200 and the caller requests 200 bytes. How many bytes should be returned? Why is reaching END alone insufficient to determine the number of valid bytes in the last block?

(iii) Complete the expressions for an arbitrary file position `pos`. How do you use the first value to locate the correct block in a FAT chain?

```
unsigned long file_block_index = __________;
unsigned long offset_in_block = __________;
```

**Solution**

(i) Read 12 bytes at offsets 500–511 of data block 2 (device block 5), then 512 bytes from data block 7 (device block 10), then 76 bytes from data block 4 (device block 7). The new position is $500 + 600 = 1100$. File order follows the chain, not ascending device addresses.

(ii) Return $1300 - 1200 = 100$ bytes, all from the final block. Its offset is $1200 \bmod 512 = 176$. END identifies the final allocated block, but the file size identifies the valid portion of it. The file uses three blocks (1536 bytes of data-block capacity), only 1300 of which are file contents.

(iii) Use `pos / 512` and `pos % 512`. Start at `start_blk` and follow the FAT link `file_block_index` times. A file-block index is an ordinal within the file, not a data block address.

(c) **Growing and deleting**: A 500-byte file starts at data block 5, and `FAT[5] = NGFS_BLOCK_END`. Data block 6 is free. Assume there is enough directory space and all operations succeed.

(i) Append 40 bytes, allocating block 6 if needed. How many bytes go into each block? Give the new FAT entries and directory-entry size.

(ii) The following deletion loop is incorrect. Identify the bug and rewrite the loop. Assume a valid, nonempty chain and ignore synchronization here.

```
while (b != NGFS_BLOCK_END) {
    FAT[b] = NGFS_BLOCK_FREE;
    b = FAT[b];
}
```

(iii) Besides freeing the chain, how does the lecture's NGFS deletion method update the root directory? Must it erase the old file bytes? If a FAT or directory block was modified through the cache, how should it be released?

**Solution**

(i) The original block has 12 spare bytes: write 12 there and 28 into block 6. Set `FAT[5] = 6`, `FAT[6] = NGFS_BLOCK_END`, and the file size to 540 bytes. The starting block remains 5.

(ii) The code destroys the next-block link before reading it, so it follows FREE (0) instead of the chain. Save the link first:

```
while (b != NGFS_BLOCK_END) {
    uint32_t next = FAT[b];
    FAT[b] = NGFS_BLOCK_FREE;
    b = next;
}
```

(iii) Replace the removed entry with the last entry (unless already last), and shrink the directory's size by one entry. Old file bytes need not be erased for logical deletion. Release each successfully fetched cache block exactly once, marking modified blocks dirty so updates can reach the device.